

SDD

Dokument softwarového návrhu (SDD)

Angela's Cakes – Webová aplikace pro objednávání dortů

Verze: 1.0

Datum: 21. května 2026

Autor: Vlad Balan

Obsah

1. Úvod
 2. Architektura systému
 3. Návrh backendu
 4. Návrh frontendu
 5. Databázový návrh
 6. Bezpečnostní návrh
 7. REST API referenční přehled
-

1. Úvod

1.1 Účel dokumentu

Tento dokument popisuje softwarový návrh aplikace Angela's Cakes. Zachycuje architekturní rozhodnutí, strukturu vrstev, databázový model, návrh bezpečnosti a přehled REST API. Cílová skupina jsou vývojáři a architekti systému.

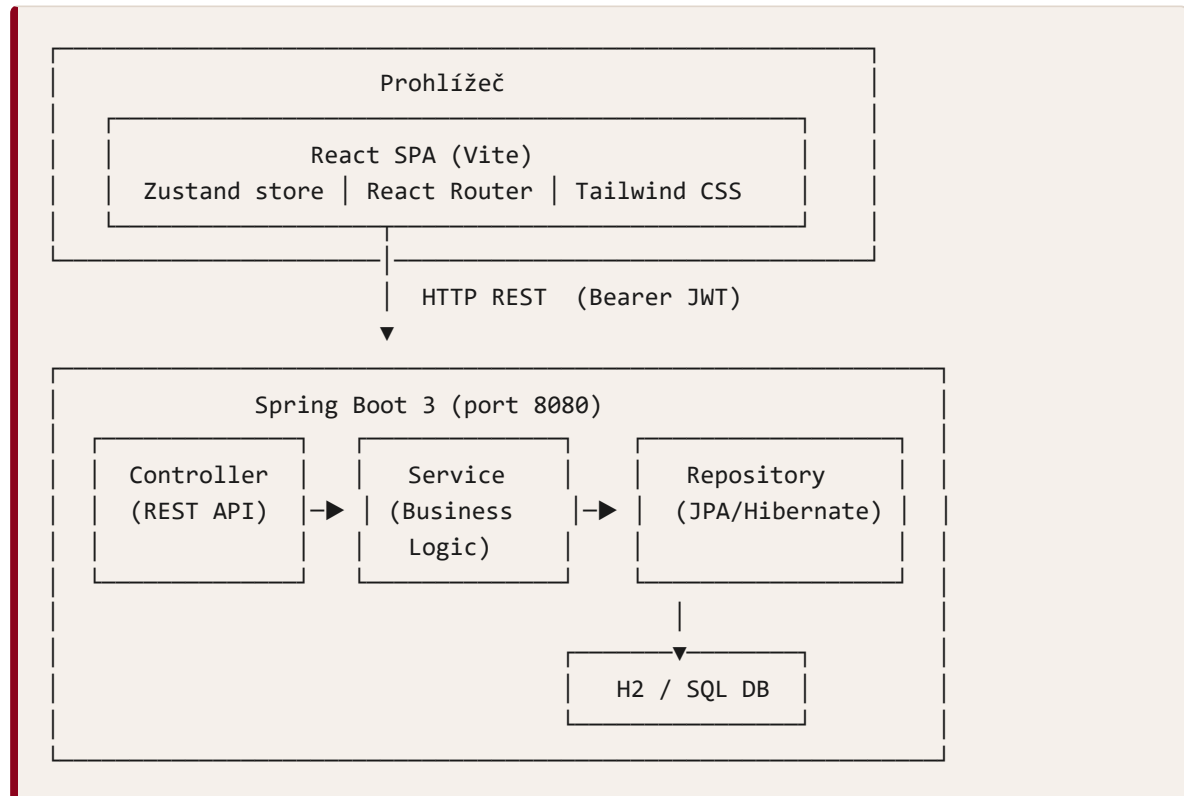
1.2 Přehled systému

Aplikace se skládá ze dvou nezávisle nasaditelných komponent:

- **Backend:** Spring Boot 3 REST API server, Java 21.
- **Frontend:** React 19 SPA (Vite), komunikuje s backendem výhradně přes HTTP API.

2. Architektura systému

2.1 Celková architektura



2.2 Komunikační tok

1. Uživatel provede akci v prohlížeči.
2. React komponenta zavolá funkci ze Zustand store nebo přímo přes `axios`.
3. Axios přidá `Authorization: Bearer <token>` hlavičku automaticky (interceptor).
4. Backend přijme požadavek, ověří JWT přes `JwtFilter`.
5. Controller deleguje na Service, Service na Repository.
6. Odpověď (JSON) se vrátí frontendu.
7. Stav se uloží do Zustand store nebo lokálního `useState`.

3. Návrh backendu

3.1 Technologický zásobník

Technologie	Verze	Účel
Java	21	Programovací jazyk
Spring Boot	3.x	Aplikační framework
Spring Security	6.x	Autentizace a autorizace
Spring Data JPA	3.x	Perzistence dat
Hibernate	6.x	ORM
H2 Database	2.x	Vývojová databáze (in-memory / file)
Lombok	–	Redukce boilerplate kódu
jjwt (JJWT)	–	Generování a validace JWT tokenů
Springdoc OpenAPI	–	Swagger UI dokumentace

3.2 Balíčková struktura

```

com.example.angelascakes/
├── config/
│   ├── DataInitializer.java      # Seed dat při startu (admin účet, typy dortů)
│   ├── GlobalExceptionHandler.java # Centralizované zpracování výjimek
│   ├── SecurityConfig.java      # Spring Security + CORS konfigurace
│   ├── SwaggerConfig.java      # OpenAPI konfigurace
│   └── WebConfig.java           # Statické assety (/images/**)
├── controller/
│   ├── AuthController.java      # POST /auth/register, POST /auth/login
│   ├── CakeController.java      # CRUD dortů + dostupnost
│   ├── CakeSizeController.java  # CRUD velikostí dortů
│   ├── CakeTypeController.java  # CRUD typů dortů
│   ├── DecorationController.java # CRUD dekorací
│   ├── FlavorController.java    # CRUD příchutí
│   ├── FileUploadController.java # Nahrávání obrázků
│   ├── OrderController.java     # Objednávky (zákazník + admin)
│   └── UserProfileController.java # Profil a adresa zákazníka
├── dto/                          # Data Transfer Objects (Request/Response)
├── entity/                       # JPA entity (Cake, Order, User, ...)
├── exception/
│   └── ResourceNotFoundException.java # Vyvolá HTTP 404
├── repository/                  # Spring Data JPA repozitáře
├── security/
│   ├── JwtFilter.java           # Validace JWT u každého požadavku
│   ├── JwtUtil.java            # Generování a parsování JWT
│   └── UserDetailsServiceImpl.java # Načtení uživatele pro Spring Security
└── service/                     # Obchodní logika

```

3.3 Vrstvový návrh (Controller–Service–Repository)

Controller

- Přijímá HTTP požadavky a mapuje je na metody.
- Provádí autorizaci přes `@PreAuthorize`.
- Deleguje veškerou logiku na Service.
- Vrací `ResponseEntity<DTO>`.

Service

- Obsahuje veškerou obchodní logiku.
- Načítá entity přes Repository, mapuje je na DTO.
- Vyvolává `ResourceNotFoundException` (→ HTTP 404) pro neexistující zdroje.
- Validuje oprávnění na úrovni dat (např. zákazník smí vidět jen vlastní objednávky).

Repository

- Rozšiřuje `JpaRepository` nebo `PagingAndSortingRepository`.
- Obsahuje custom JPQL dotazy pro filtrování (např. `searchCakes`, `searchAllCakes`).
- Pro zákazníka filtruje pouze dostupné dory (`available = true`).

3.4 Zpracování výjimek

```
ResourceNotFoundException → GlobalExceptionHandler → HTTP 404
RuntimeException           → GlobalExceptionHandler → HTTP 400
MethodArgumentNotValidException → HTTP 400 (Bean Validation)
```

`GlobalExceptionHandler` je anotován `@RestControllerAdvice` a zachycuje výjimky globálně. Vrací JSON objekt s polem `message`.

3.5 Nahrávání souborů

`FileUploadService` ukládá obrázky na disk do adresáře `uploads/images/`. `WebConfig` konfiguruje statické servírování z tohoto adresáře na cestě `/images/**`. Vracená URL je relativní cesta použitelná přímo jako `src` obrázku na frontendu.

3.6 DataInitializer

Spustí se při každém startu aplikace jako `CommandLineRunner`. Pokud neexistuje administrátorský účet, vytvoří jej:

- E-mail: `admin@angelascakes.com`

- Heslo: `admin123` (bcrypt hash)
- Role: `ADMIN`

Pokud tabulka `cake_types` je prázdná, vytvoří výchozí typy: Regular, Birthday, Cheesecake, Wedding.

4. Návrh frontendu

4.1 Technologický zásobník

Technologie	Verze	Účel
React	19	UI framework
Vite	5.x	Build nástroj a dev server
React Router DOM	7	Klientské směrování
Zustand	5	Globální správa stavu
Axios	–	HTTP klient
Tailwind CSS	4 (CSS-first)	Stylování
Lucide React	–	Ikonová sada

4.2 Adresářová struktura

```

src/
├── api/
│   └── axios.js          # Axios instance s interceptorem (přidání JWT)
├── components/
│   ├── admin/           # AdminSidebar, Field, ImageUploader, CakeSizesEditor, M
│   ├── catalog/         # CakeCard, CatalogFilterSidebar
│   ├── cart/            # CartItem
│   ├── customer/        # ProfileCard, DeliveryAddressCard
│   ├── layout/          # Navbar, AdminSidebar, CustomerSidebar, Sidebar
│   ├── order/           # OrderCard, ActiveOrderCard, StatusBadge, StatusTimelin
│   └── ui/              # Generické UI (button, scroll-area)
├── pages/
│   ├── auth/            # LoginPage, RegisterPage
│   ├── admin/           # AdminCakesPage, AdminCakeDetailPage, AdminOrdersPage,
│   │                   # AdminFlavorsPage, AdminDecorationsPage, AdminCakeTypes
│   ├── customer/        # CatalogPage, CakeDetailPage, CartPage, OrderConfirmati
│   │                   # DashboardPage, SettingsPage, OrderHistoryPage, OrderDe
│   └── NotFoundPage.jsx
├── store/
│   ├── authStore.js      # JWT token, uživatelský objekt (Zustand + persist)
│   ├── cartStore.js      # Obsah košíku (Zustand + persist)
│   └── notificationStore.js # Počet nepřečtených objednávek pro admina
├── utils/
│   ├── format.js        # formatPrice, formatDate, formatDateTime
│   └── orderStatus.js    # STATUS_CONFIG, ORDER_STATUSES (label, barvy)
└── App.jsx              # Router, ProtectedRoute, AdminRoute

```

4.3 Správa stavu (Zustand)

authStore

Uchovává JWT token a uživatelský objekt (email, role). Perzistováno v `localStorage` klíč `auth-store`.

```

{
  token: string | null,
  user: { email: string, role: 'CUSTOMER' | 'ADMIN' } | null,
  setAuth(token, user),
  logout()
}

```

cartStore

Uchovává pole položek košíku. Perzistováno v `localStorage` klíč `cart-store`.

```
{
  items: CartItem[],
  addItem(cake, size, quantity),
  removeItem(cakeId, sizeId),
  updateQuantity(cakeId, sizeId, quantity),
  clear()
}
```

notificationStore

Počet nepřečtených objednávek pro administrátora. Není perzistován.

```
{
  unseenCount: (number, setUnseenCount(n), decrement());
}
```

4.4 Axios interceptor

Soubor `src/api/axios.js` vytváří Axios instanci s `baseUrl: 'http://localhost:8080/api'`. Request interceptor přečte token z `authStore` a vloží jej do hlavičky `Authorization: Bearer <token>`.

4.5 Směrování a ochrana cest

Definováno v `App.jsx`:

Komponenta	Podmínka	Chování při nesplnění
<code>ProtectedRoute</code>	Token existuje	Přesměrování na <code>/login</code>
<code>AdminRoute</code>	Token existuje + role = ADMIN	Bez tokenu → <code>/login</code> ; bez ADMIN role → <code>/</code>

Všechny zákaznické a administrátorské trasy jsou obaleny příslušnou ochrannou komponentou. Wildcard `*` renderuje `NotFoundPage`.

4.6 Zachování filtrů při navigaci

- **CatalogPage:** Filtry jsou zakódovány v URL search params (`?name=...&flavorId=...`). Tlačítko zpět (`navigate(-1)`) obnoví celou URL včetně filtrů.
- **AdminCakesPage, AdminOrdersPage:** Filtry a stav řazení jsou ukládány do `sessionStorage` při každé změně a inicializovány z `sessionStorage` při načtení stránky. Tím přežijí navigaci v rámci jedné session.

4.7 Design systém (Tailwind CSS v4)

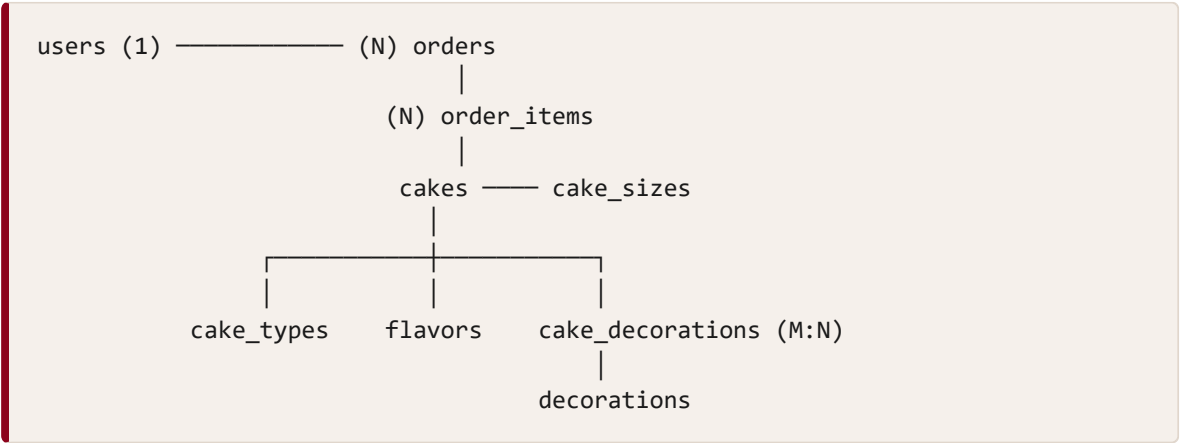
Barvy jsou definovány jako CSS custom properties v `@theme` :

Token	Hodnota	Použití
<code>--color-crimson</code>	<code>#8C031C</code>	Primární akce, ceny, aktivní prvky
<code>--color-brown</code>	<code>#3D2713</code>	Texty, nadpisy
<code>--color-teal</code>	<code>#005B48</code>	Sekundární akcenty, typ dortu
<code>--color-peach</code>	<code>#FEE3C9</code>	Tagy, pozadí karet
<code>--color-stroke</code>	<code>#DCDCDC</code>	Okraje, oddělovače
Pozadí stránek	<code>#FFF5EC</code>	Teplý krémový podklad

Typografie používá dva fonty: serif nadpisový font (`font-heading`) a bezpatkový tělo (`font-body`).

5. Databázový návrh

5.1 Entity-Relationship diagram (textový přehled)



5.2 Schéma tabulek

`users`

Sloupec	Typ	Omezení
id	BIGINT	PK, AUTO_INCREMENT
email	VARCHAR	NOT NULL, UNIQUE
password	VARCHAR	NOT NULL (bcrypt hash)
role	VARCHAR	NOT NULL ('CUSTOMER' nebo 'ADMIN')
first_name	VARCHAR	nullable
last_name	VARCHAR	nullable
address_line	VARCHAR	nullable
building_name	VARCHAR	nullable
street_name	VARCHAR	nullable
postcode	VARCHAR	nullable
city	VARCHAR	nullable

cakes

Sloupec	Typ	Omezení
id	BIGINT	PK, AUTO_INCREMENT
name	VARCHAR	NOT NULL
description	VARCHAR(500)	nullable
image_url	VARCHAR	nullable
price	DECIMAL	NOT NULL
available	BOOLEAN	NOT NULL, DEFAULT false
flavor_id	BIGINT	FK → flavors
cake_type_id	BIGINT	FK → cake_types

cake_sizes

Sloupec	Typ	Omezení
id	BIGINT	PK
cake_id	BIGINT	FK → cakes, NOT NULL
label	VARCHAR	NOT NULL (např. "S", "M", "L")
slices	INT	NOT NULL
price	DECIMAL	NOT NULL

cake_decorations (vazební tabulka M:N)

Sloupec	Typ
cake_id	BIGINT (FK → cakes)
decoration_id	BIGINT (FK → decorations)

orders

Sloupec	Typ	Omezení
id	BIGINT	PK, AUTO_INCREMENT
user_id	BIGINT	FK → users, NOT NULL
status	VARCHAR	NOT NULL (enum OrderStatus)
total_price	DECIMAL	NOT NULL
created_at	TIMESTAMP	NOT NULL
seen	BOOLEAN	NOT NULL, DEFAULT false
note	VARCHAR	nullable
delivery_first_name	VARCHAR	nullable
delivery_last_name	VARCHAR	nullable
delivery_address_line	VARCHAR	nullable
delivery_building_name	VARCHAR	nullable
delivery_street_name	VARCHAR	nullable
delivery_postcode	VARCHAR	nullable
delivery_city	VARCHAR	nullable

Poznámka k delivery polím: Adresa je při vytvoření objednávky zkopírována ze zákaznickova profilu. Tím je zajištěno, že historická objednávka ukazuje adresu platnou v době objednání, i když zákazník svůj profil spáťer změní.

order_items

Sloupec	Typ	Omezení
id	BIGINT	PK
order_id	BIGINT	FK → orders, NOT NULL
cake_id	BIGINT	reference na dort (nullable pokud dort smazán)
cake_name	VARCHAR	snímek názvu dortu v době objednání
cake_image_url	VARCHAR	snímek URL obrázku
size_label	VARCHAR	snímek velikosti
slices	INT	snímek počtu porcí
price_at_order	DECIMAL	snímek ceny v době objednání
quantity	INT	NOT NULL
total_price	DECIMAL	NOT NULL (price_at_order × quantity)

5.3 Výchozí data (seed)

Při prvním spuštění `DataInitializer` vytvoří:

- Admin uživatele: `admin@angelascakes.com` / `admin123`
- Typy dortů: Regular, Birthday, Cheesecake, Wedding

6. Bezpečnostní návrh

6.1 Autentizace (JWT)

1. Klient odešle POST `/api/auth/login` s e-mailem a heslem.
2. `AuthService` ověří heslo pomocí `BCryptPasswordEncoder`.
3. `JwtUtil` vygeneruje podepsaný JWT token (HMAC-SHA256) s claimem `sub` = email uživatele.
4. Token se vrátí klientovi; klient jej uloží do `localStorage`.

5. Při každém dalším požadavku `JwtFilter` extrahuje token z hlavičky `Authorization: Bearer`, validuje podpis a expiraci, a nastaví `SecurityContext`.

6.2 Autorizace

- Metoda-level security je povolena anotací `@EnableMethodSecurity`.
- Každý controller endpoint má explicitní `@PreAuthorize("hasRole('ADMIN')")` nebo `@PreAuthorize("hasAnyRole('CUSTOMER', 'ADMIN')")`.
- Administrátorské endpointy jsou tímto zcela odděleny od zákaznických.

6.3 CORS

CORS je konfigurován v `SecurityConfig.corsConfigurationSource()`:

- Povolený origin: `http://localhost:5173`
- Povolené metody: GET, POST, PUT, PATCH, DELETE, OPTIONS
- Credentials: povoleny (kvůli JWT v hlavičce)

6.4 Ochrana hesla

Hesla jsou hashována algoritmem **BCrypt** (výchozí faktor 10) pomocí Spring Security `BCryptPasswordEncoder`. Heslo v plain textu není nikde ukládáno.

7. REST API referenční přehled

7.1 Autentizace

Metoda	URL	Oprávnění	Popis
POST	<code>/api/auth/register</code>	public	Registrace nového zákazníka
POST	<code>/api/auth/login</code>	public	Přihlášení, vrátí JWT token

7.2 Dorty

Metoda	URL	Oprávnění	Popis
GET	/api/cakes	authenticated	Seznam dostupných dortů (filtrování, stránkování)
GET	/api/cakes/{id}	authenticated	Detail dortu
GET	/api/cakes/admin/all	ADMIN	Všechny dorty (včetně nedostupných)
POST	/api/cakes	ADMIN	Vytvoření nového dortu
PUT	/api/cakes/{id}	ADMIN	Aktualizace dortu
PATCH	/api/cakes/{id}/availability	ADMIN	Přepnutí dostupnosti dortu

7.3 Velikosti dortů

Metoda	URL	Oprávnění	Popis
GET	/api/cake-sizes/cake/{cakeId}	authenticated	Velikosti konkrétního dortu
POST	/api/cake-sizes	ADMIN	Přidání velikosti
PUT	/api/cake-sizes/{id}	ADMIN	Aktualizace velikosti
DELETE	/api/cake-sizes/{id}	ADMIN	Smazání velikosti

7.4 Atributy (příchutě, dekorace, typy dortů)

Metoda	URL	Oprávnění	Popis
GET	/api/flavors	authenticated	Seznam příchutí
POST	/api/flavors	ADMIN	Přidání příchutě
PUT	/api/flavors/{id}	ADMIN	Přejmenování příchutě
DELETE	/api/flavors/{id}	ADMIN	Smazání příchutě
GET	/api/decorations	authenticated	Seznam dekorací
POST/PUT/DELETE	/api/decorations/{id}	ADMIN	Správa dekorací
GET	/api/cake-types	authenticated	Seznam typů dortů
POST/PUT/DELETE	/api/cake-types/{id}	ADMIN	Správa typů dortů

7.5 Objednávky

Metoda	URL	Oprávnění	Popis
POST	/api/orders	CUSTOMER / ADMIN	Vytvoření objednávky
GET	/api/orders/my	CUSTOMER / ADMIN	Objednávky přihlášeného uživatele
GET	/api/orders/my/{id}	CUSTOMER / ADMIN	Detail vlastní objednávky
POST	/api/orders/my/{id}/cancel	CUSTOMER / ADMIN	Zrušení vlastní objednávky
GET	/api/orders	ADMIN	Všechny objednávky
GET	/api/orders/{id}	ADMIN	Detail libovolné objednávky
PUT	/api/orders/{id}/status	ADMIN	Změna stavu objednávky
PUT	/api/orders/{id}/seen	ADMIN	Označení objednávky jako přečtené
GET	/api/orders/unseen-count	ADMIN	Počet nepřečtených objednávek

7.6 Uživatelský profil

Metoda	URL	Oprávnění	Popis
GET	/api/profile	authenticated	Načtení profilu
PUT	/api/profile	authenticated	Aktualizace jména
PUT	/api/profile/address	authenticated	Aktualizace doručovací adresy

7.7 Nahrávání souborů

Metoda	URL	Oprávnění	Popis
POST	/api/upload/image	ADMIN	Nahrání obrázku dortu

Obrázky jsou staticky dostupné na `/images/{filename}`.